运行时动态链接：/bin/echo 如何调用到 libc.so 里的 exit()

内核 exec（①–③） → 用户态 ld-xv6（④–⑥） → 程序运行（⑦–⑧） ・ 地址取自 readelf，libc 基址 0x9000

内核 / 磁盘

内核 exec() EL1

读 ELF 头和程序头

PT_INTERP → /lib/ld-xv6.so

eret → EL0

fs.img（磁盘文件）

/bin/echo fs.img

ET_DYN · INTERP · NEEDED libc.so

/lib/ld-xv6.so fs.img

ET_DYN · 无重定位 · entry 0x750

/lib/libc.so fs.img

write 0x3250 · exit 0x3cc0

strlen 0x3de0 · 71 JUMP_SLOT

进程虚拟地址空间（地址向下增长，未按比例）

/bin/echo • LOAD #1 • R-X 0x0000–0x0358

.hash .dynsym .dynstr [2] exit UND

.rela.plt 0x14a0 JUMP_SLOT sym=2

0x2a8 exit@plt: ldr x17,[0x14a0]; br x17

0x2c8 main … 0x304: bl exit@plt

/bin/echo • LOAD #2 • RW- 0x1358–0x14b0

.dynamic NEEDED libc.so · SYMTAB · HASH · JMPREL

.got.plt 0x1498 write 0x278 → 0xc250

0x14a0 exit 0x278 → 0xccc0

0x14a8 strlen 0x278 → 0xcde0

/lib/ld-xv6.so（AT_BASE） 0x2000–0x3d20

0x2750 ldso_start = 0x2000 + e_entry 0x750

parse_dynamic · load_libc · relocate · lookup

保护页 0x4000–0x5000

去掉 PTE_U：栈溢出立即缺页

用户栈（4 页） 0x5000–0x9000

sp → argc 由 x0 传 · argv[] · NULL · envp NULL

auxv AT_PHDR=0x40 AT_PHENT=56 AT_PHNUM=5

AT_BASE=0x2000 AT_ENTRY=0x2c8 AT_NULL

argv 字符串 "echo" "hi" …（靠近栈顶 0x9000）

libc.so • 代码 • R-X（sbrk 所得） 0x9000–0x122ba

.hash .dynsym .dynstr exit: st_value 0x3cc0

0xc250 write = 0x9000 + 0x3250

0xccc0 exit = 0x9000 + 0x3cc0 … svc #0

0xcde0 strlen = 0x9000 + 0x3de0

libc.so • 数据 • RW- 0x132c0–0x13650

.dynamic · .got.plt 71 槽（ld-xv6 先填 libc 自己的）

写 0xccc0

步骤

1 内核 exec 装主程序

读 /bin/echo 的程序头：

LOAD#1 → 0x0，LOAD#2 → 0x1358

2 装解释器、建栈

PT_INTERP → 把 ld-xv6.so 装到

PGROUNDUP(0x14b0) = 0x2000;

保护页 + 4 页栈，压 argv/envp/auxv

3 eret 进入 ld-xv6

PC = 0x2000 + 0x750 = 0x2750

SP = 栈上参数表，EL0

4 ld-xv6 找到 _echo 的 .dynamic

auxv 取 AT_PHDR=0x40 → 扫程序头

→ PT_DYNAMIC 0x1358 → 读出

NEEDED / SYMTAB / HASH / JMPREL

5 装入 libc.so

open + sbrk → 基址 0x9000

pread 两个 PT_LOAD

先填 libc 自己的 71 个 GOT 槽

6 绑定 _echo 的 GOT

.rela.plt: sym=2 → "exit"（UND）

→ 查 libc .hash/.dynsym = 0x3cc0

→ 写 [0x14a0] = 0x9000+0x3cc0

= 0xccc0（write/strlen 同理）

7 跳到 main

AT_ENTRY = 0x2c8

main(argc, argv)

8 调用 exit()

bl exit@plt (0x2a8)

→ ldr x17,[0x14a0] = 0xccc0

→ br x17 → libc exit → svc #0

①–③ 内核 exec：只装载、建栈、跳到解释器，不查符号

④–⑥ ld-xv6（EL0）：装 libc、查 .dynsym、写 GOT

⑦–⑧ 程序运行：每次调用都经 PLT 读 GOT 跳进 libc

数据：readelf -l/-r/--dyn-syms user/_echo、user/ld-xv6.so、user/libc.so；USTACKPAGES = 4